

iSAQB Module: Formal Methods - Satisfiability Modulo Theories

Active Group GmbH

Created: 2026-10-07 Wed 11:17

Bradycardia

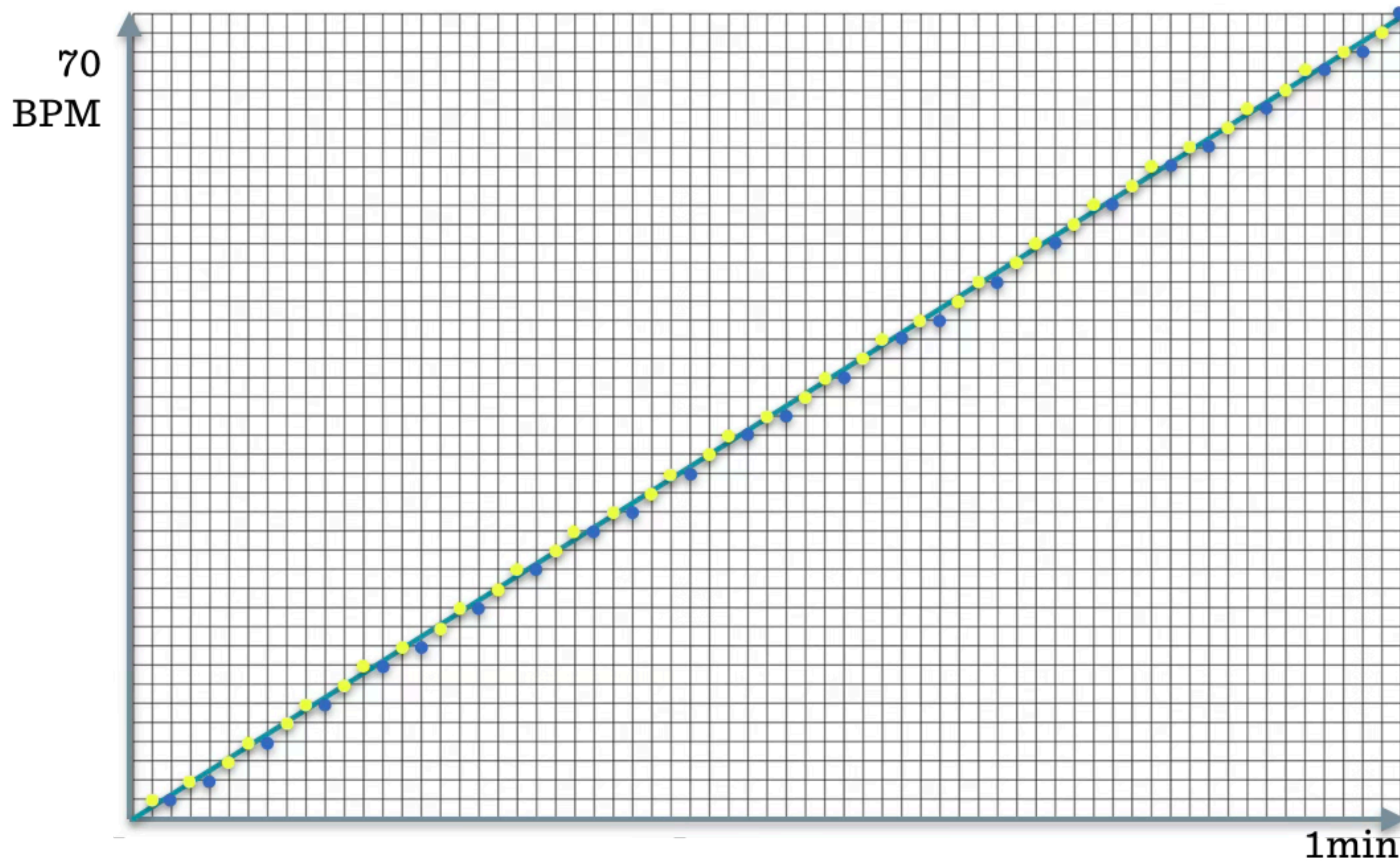


Constraints & Requirements

- long battery life
- simple hardware
- correct algorithm



Pacemaker



$$y = \frac{dy}{dx}x$$

Bresenham's Algorithm

```
plotLine(x0, y0, x1, y1)
  dx = x1 - x0
  dy = y1 - y0
  D = 2*dy - dx
  y = y0

  for x from x0 to x1
    plot(x, y)
    if D > 0
      y = y + 1
      D = D + (2 * (dy - dx))
    else
      D = D + 2*dy
    end if
```

https://en.wikipedia.org/wiki/Bresenham%27s_line_algorithm

Satisfiability Modulo Theories (1)

- Good: The validity of propositional logic expressions can be decided in finite time ("SAT is decidable").
- Bad: The toolbox of propositional logic is very restricted. The only sort is Booleans and there are no quantifiers.
- First order logic is more expressive, but it is no longer decidable.
- Could there be something in the middle? Yes!

Satisfiability Modulo Theories (2)

- SMT enriches SAT with *theories*, i.e. new sorts and operations
 - Integer arithmetic
 - Real arithmetic
 - Arrays
 - Bit vectors
 - Strings
 - Lists
 - ...
- These sorts and operations can be described ("axiomatized") in first order logic.
- Some (combinations) of these theories are still decidable.
- Some combinations of these theories are not decidable in general, but best-effort approaches often *do* terminate
- Most theories come with efficient decision procedures (in the average case)
- See <https://dl.acm.org/doi/pdf/10.1145/1995376.1995394> for an introduction

Integrating SAT solvers and theory solvers

- How to solve SMT formulae (efficiently)?

Consider the SMT formula that consists of a conjunction of eight disjunctions that each contain either an inequality directly or a disjunction of inequalities.

```
t11 ≥ 0 ∧ t12 ≥ t11 + 2 ∧ t12 + 4 ≤ 6 ∧  
t21 ≥ 0 ∧ t22 ≥ t21 + 3 ∧ t22 + 1 ≤ 6 ∧  
(t11 ≥ t21 + 3 ∨ t21 ≥ t11 + 2) ∧  
(t12 ≥ t22 + 1 ∨ t22 ≥ t12 + 4)
```

We call the inequalities *atoms*.

We check this formula for satisfiability in two steps:

1. Run a SAT solver on the abstracted formula where we replace atoms with fresh variables, for example with the Davis-Putnam-Logeman-Loveland (DPLL) algorithm.
2. Refine these SAT solutions with the help of theory solvers for difference arithmetic.

SAT solving: Davis-Putnam-Logeman-Loveland (DPLL)

- Given a SAT formula in conjunctive normal form (CNF, a conjunction of disjunctions, as above)
- Choose (heuristically) an unassigned variable/atom X
 - Assume the variable X to be true
 - Simplify the CNF formula accordingly
 - If the formula contains a conflicting clause (all atoms are false):
 - Assume the variable X to be false
 - Simplify accordingly
 - If the formula contains a conflicting clause: backtrack
 - If the formula does not contain a conflicting clause: continue with the next unassigned variable
- In the end we either find a complete variable assignment that satisfies the formula, or we backtracked to the very beginning with no other choices to make.

A theory solver for difference arithmetic (1)

- Difference arithmetic is a fragment of linear arithmetic, where predicates are restricted to be of the form $t - s \leq c$ and where t and s are variables and c a numeric constant. In our example all the atoms are of this form or can be transformed into this form.

A theory solver for difference arithmetic (2)

- A set of difference arithmetic atoms can be checked efficiently for satisfiability by searching for negative cycles in weighted directed graphs.
- Each variable corresponds to a node
- Each inequality $t - s \leq c$ corresponds to an edge from s to t with weight c .
- Cycles with negative weight sum mean: not satisfiable!

Integration of SAT and theory solving

- Solve the abstracted formula with a SAT solver
- The SAT solver either finds the abstracted formula unsatisfiable ...
 - ... then so is the original formula
- ... or it finds a model (assignment of atoms to true or false)
- ... which leads to a set of difference arithmetic atoms that we can check with the theory solver.

Example (1)

We begin with this SMT formula:

```
t11 ≥ 0 ∧ t12 ≥ t11 + 2 ∧ t12 + 4 ≤ 6 ∧  
t21 ≥ 0 ∧ t22 ≥ t21 + 3 ∧ t22 + 1 ≤ 6 ∧  
(t11 ≥ t21 + 3 ∨ t21 ≥ t11 + 2) ∧  
(t12 ≥ t22 + 1 ∨ t22 ≥ t12 + 4)
```

Example (2)

The abstracted SAT formula is:

```
A ∧ B ∧ C ∧  
D ∧ E ∧ F ∧  
(G ∨ H) ∧  
(I ∨ J)
```

Example (3)

DPLL suggests that we obtain a model by setting all variables to `true`. This leads to the following set of atoms:

```
t11 ≥ 0
t12 ≥ t11 + 2
t12 + 4 ≤ 6
t21 ≥ 0
t22 ≥ t21 + 3
t22 + 1 ≤ 6
t11 ≥ t21 + 3
t21 ≥ t11 + 2
t12 ≥ t22 + 1
t22 ≥ t12 + 4
```

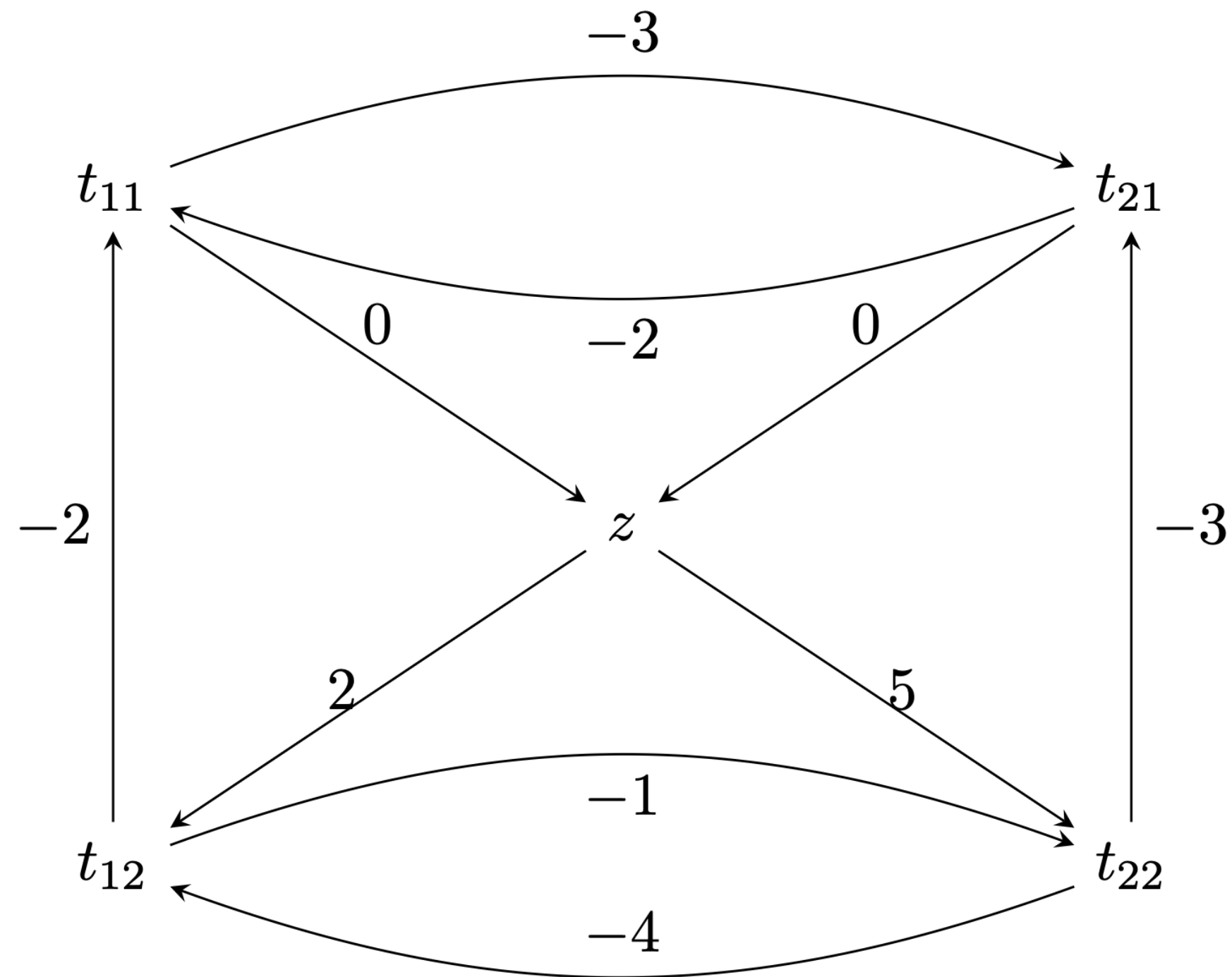
Example (4)

We transform these atoms into the form $t - s \leq c$. For the constant zero we introduce a special variable z .

$$\begin{array}{rcl} z & - & t_{11} \leq 0 \\ t_{11} & - & t_{12} \leq -2 \\ t_{12} & - & z \leq 2 \\ z & - & t_{21} \leq 0 \\ t_{21} & - & t_{22} \leq -3 \\ t_{22} & - & z \leq 5 \\ t_{21} & - & t_{11} \leq -3 \\ t_{11} & - & t_{21} \leq -2 \\ t_{22} & - & t_{12} \leq -1 \\ t_{12} & - & t_{22} \leq -4 \end{array}$$

Example (5)

Draw the graph



There is a negative cycle! This means that all atoms set to true is *not* a model for the initial formula. We must continue with the SAT solver.

Logic fragments

When formulae are restricted to certain subsets of the entire formula space, more efficient decision procedures can be employed. These subsets are often referred to as fragments.

For example, a formula that uses only linear integer arithmetic (multiplications are only by known constants) can be checked very efficiently (LIA). In addition, if the formula doesn't even contain any quantifiers (QF_{LIA}), it can be solved even more efficiently. In contrast, formulae that use full multiplication (NIA) become harder to check.

See: [SMT-LIB Logics](#)

SMT-LIB2

- SMT-LIB2 is the standardized language of SMT
- <https://smt-lib.org>
- SMT-LIB2 was designed to be a format for interchange ...
- ... but it's actually a nice programming language as well.

SMT-LIB2 example (1)

```
(declare-const a Int)
(assert (> a 3))
(check-sat)
(get-model)
```

```
sat
(
  (define-fun a () Int
    4)
)
```

- SMT-LIB2 uses an S-expression based syntax. That means every compound expression is of the form ...
 - (operator operand-1 operand-2 ... operand-n) where the operator and the operands are expressions in turn

SMT-LIB2: declaring constants

- Constants can be declared with `declare-const`, which takes a name and a **sort** such as:
 - `Int`
 - `Real`
 - `Bool`
 - ...

```
(declare-const x Int)
(declare-const y Real)
(declare-const z Bool)
```

SMT-LIB2: Asserting constraints

- Constraints can be asserted with `assert`, which takes a boolean expression as an argument

```
| (assert (= (+ x y) z))
```

SMT-LIB2: Checking satisfiability

- `check-sat` instructs the smt solver to check for satisfiability.

```
(declare-const x Int)
(assert (> x 3))
(assert (< x 5))
(check-sat)
```

- Save this as a file ``assert.smt2`` and run it with Z3 by invoking `z3 assert.smt2`
- The result should be a simple `sat` meaning this program is satisfiable
- Try making the asserts express a contradiction. The result will be `unsat`.

SMT-LIB2: push and pop

- You can use push and pop to keep assertions from interfering with one other
- This is useful for scratchpad-style programming

```
(declare-const x Int)
```

```
(push)
(assert (> x 3))
(check-sat)
(pop)
```

```
(push)
(assert (< x 3))
(check-sat)
(pop)
```

Result:

```
sat
sat
```

SMT-LIB2: Declaring data types

- `declare-datatypes` takes ...
 - ... a name for a new sort (`Line`)
 - ... a list of constructor definitions, each of which takes ...
 - ... a name for the constructor (`mk-line`)
 - ... a set of fields with a field name (`line-dx` and `line-dy`) and their corresponding sorts (`Int`)

```
(declare-datatype Line ((mk-line (line-dx Int)
                                (line-dy Int))))
```

SMT-LIB2: Defining functions

- `define-fun` takes ...
 - ... a function name
 - ... a list of arguments with names and sorts
 - ... a sort for the return value
 - ... and an expression for the function body

```
(define-fun line-valid ((l Line))
  Bool
  (and
    (> (line-dx l) 0)
    (>= (line-dx l)
        (line-dy l))
    (>= (line-dy l)
        0)))
```

Z3

- The Z3 Theorem Prover is an open-source SMT solver developed by Microsoft
- Repo: <https://github.com/z3prover/z3>

```
% z3 some-smt-lib2-file.smt2
```

Exercise: Preliminaries

Using the division operator `/`, and the conversion function `to_real`, define a function that describes the slope of a given `Line`.

Using the multiplication operator `*`, define a function that calculates the exact (real) value for a given line at a given position on the x-axis.

Define a data type `State-1` for the algorithm state consisting of a `Line` and an `x` coordinate. Define a function `state-1-valid` that checks whether a given `State-1` is valid, i.e. whether the line is a valid line and the `x` coordinate is greater or equal zero.

Exercise: Reference algorithm (1)

Given the function `round`, which takes a real number and returns the closes integer, and the following framework for a line drawing algorithm:

```
(define-fun-rec draw-1- ((st State-1) (todo Int))
  (List Int)
  (ite (<= todo 0)
    ;; done
    nil
    ;; recurse
    (insert (state-1-y st)
            (draw-1- (step-1 st) (- todo 1))))))

(define-fun draw-1 ((l Line) (todo Int))
  (List Int)
  (draw-1- (mk-state-1 l 0) todo))
```

Exercise: Reference algorithm (2)

Fill in the blanks, i.e.

- Define the extractor function `state-1-y` that returns the integer valued (rounded) `y` value for a given `State-1` value.
- Define the step function `step-1` which takes a value of sort `Line` and returns a value of sort `Line` that corresponds to the next step in the iteration.
- Show that `step-1` preserves validity of state, i.e. if `state-1-valid` holds for the input, then `state-1-valid` holds for the output as well.